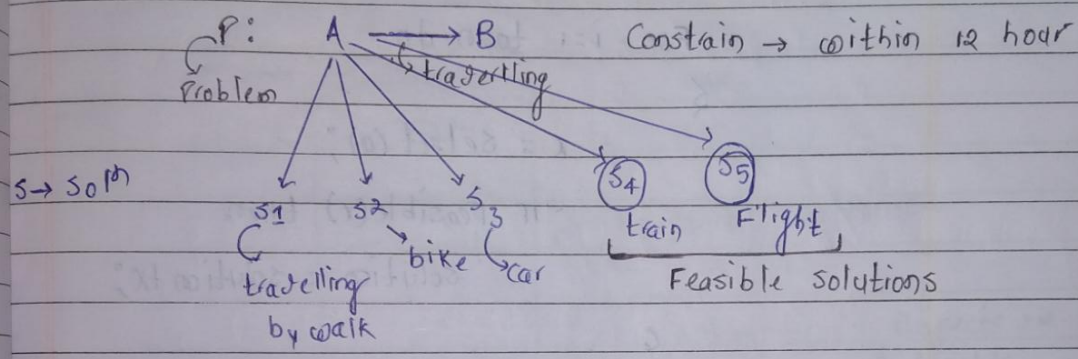
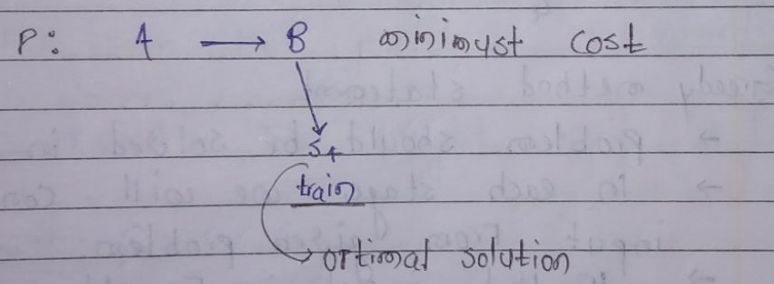
Greedy Method

It is one of strategy to solve the problem

This method is used for solving Optimisation problem
either Min^m or Max^m





** For any problem there is definitely only one optimal sol^n

1. Greedy method
2. Dynamic programming
3. Branch and Bound

* Above three strategies are used to solve optimisation problem. Approach is different for solving problem by different strategies

General method of Greedy

$n=5$

a	a_1	a_2	a_3	a_4
	1	2	3	4

```
Algorithm Greedy(a,n)
{
    For i=1 to n do
    {
        x = select(a);
        If Feasible(x) then
            solution = solution + x;
    }
}
```

Greedy method statement

→ problem should be solved in stages
→ In each stage we will consider input from given problem
→ If that input is Feasible, we will include it into solution
→ by including all those Feasible we will get optimal solution.

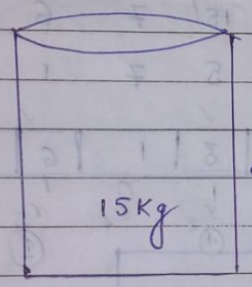
ex:-

For selecting one best person among 1000 a
(i) written Test
(ii) Technical Test / Group Discussion
(iii) Technical interview / HR interview

Knapsack Problem

$n = 7$
$m = 15$

Objects: O	1	2	3	4	5	6	7
Profits: P	10	5	15	7	6	18	3
Weights: W	2	3	5	7	1	4	1



→ Knapsack Container

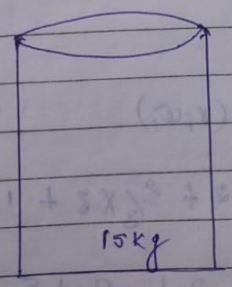
Assumption :- objects are divisible

Fill such that profit is Maximized

→ Constraint → Total weight should be less than 15 kg

Sol^n :- x $(x_1\ x_2\ x_3\ x_4\ x_5\ x_6\ x_7)$

highest profit
lowest weight



→ We should select object having highest Profit per Kg weight

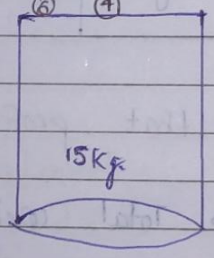
$\frac{P}{w}$:	5	1.3	3	1	6	4.5	3

Greedy method:-

First you decide how you are going select an input & go on select input one by one

P	10	5	15	7	6	18	3
w	2	3	5	7	1	4	1

P/w	5	1.3	3	1	6	4.5	3
quantity →	1	2/3	1	0	1	1	1
	x_1	x_2	x_3	x_4	x_5	x_6	x_7
	2	6	4		1	3	5



15 - 1 = 14 kg
14 - 2 = 12
12 - 4 = 8
8 - 5 = 3
3 - 1 = 2
* 2 - 2 = 0 ✗

In 1.3 kg out
I will take

$\Sigma(x_i w_i)$

weight Profit → $1 \times 2 + \frac{2}{3} \times 3 + 1 \times 5 + 0 \times 7 + 1 \times 1 + 1 \times$

$= 2 + 2 + 5 + 0 + 1 + 4 + 1 = 15$ kg

$\Sigma x_i P_i$

Profit → $1 \times 10 + \frac{2}{3} \times 5 + 1 \times 15 + 1 \times 6 + 1 \times 18 + 1$

$= 54.6$

0.1 Knapsack problem

→ it is Container loading with indivisible things [you can't take part of it]

Job sequencing with deadlines

$n = 5$

Jobs	J_1	J_2	J_3	J_4	J_5
Profits	20	15	10	5	1
deadlines	2	2	1	3	3

↳ 3 hrs

Assume → each job take one unit of time on Completion

Job → [Machine] → 1 unit

Complete each job set of jobs done within their deadline such that profit is Maximized.

→ Optimisation problem
↓
Greedy Method

Greedy method
↓
always select max one & check slot is empty or not

	0	1	2	3
AM	9	10	11	12
	J_2	J_1		J_4

First gone to Max^m

$\{ J_2, J_1, J_4 \}$

Ans → $J_1 \to J_2 \to J_4$
or
$J_2 \to J_4 \to J_4$ having same deadline

0 - 1 - 2 - 3

Job consider	slot assign	solution	Profit
-	-	0	0
J_1	[1,2]	J_1	20
J_2	[0,1],[1 2]	J_1, J_2	20+15
J_3 X	[0,1], [1,2]	J_1, J_2.	
J_4	[0,1] [1,2] [2,3]	J_1, J_2, J_4	20+15
J_5 X	"	"	40

Optimal

A

3

8

12

20

m

List → A

sizes → 6

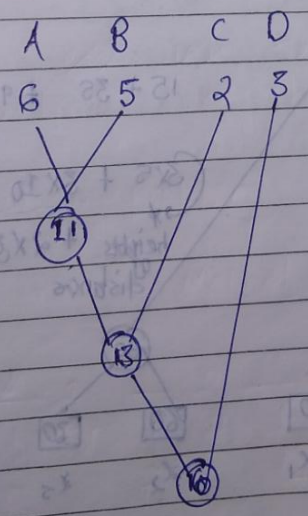
Find op

merged

Optimal Merge pattern

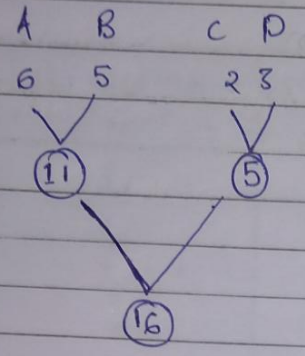
A	B	C	
3	5	3	
8	9	5	
12	11	8	
20	16	9	$\Theta(m+n)$
m	n	11	8
		12	
		16	
		20	

List → A B C D
sizes → 6 5 2 3

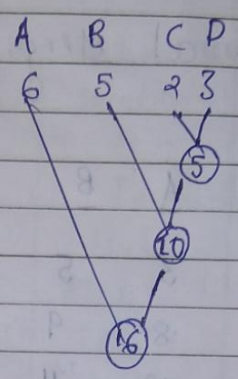


11+13+16 steps
= 40

Find optimal way so that, it can be merged into least steps



$11 + 5 + 16 = 32$



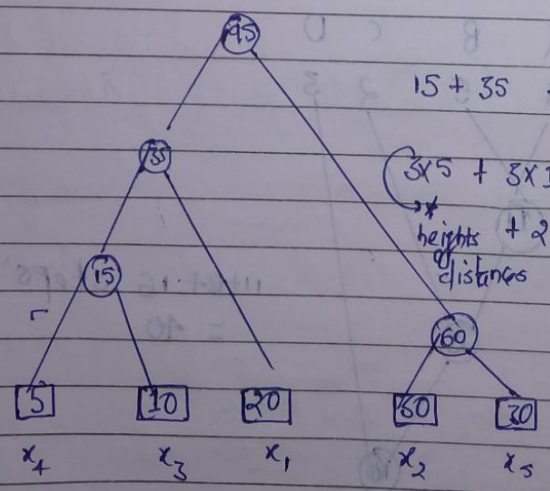
$16 + 10 + 5 = 31$

Greedy method

→ always select a pair of small siz list to get best result

2.

list →	x_1	x_2	x_3	x_4	x_5
sizes →	20	30	10	5	30



$15 + 35 + 95 + 60 = 205$

$3 \times 5 + 3 \times 10 + 2 \times 20 + 2 \times 30 + 2 \times 30 = 205$

* heights distances

$\sum d_i * x_i$

Huffman (

↓

it is (

the size

Message → B

=

1.

A

B

C

D

E

Huffman Coding

↓

it is compression technique used for reducing the size of data or message.

Message → B CC ABBDDAECCBBAEDDCC

length = 20

ASCII – 8-bits

A	65	01000001
B	66	01000010
C	67	⋮
D	68	⋮
E	69	⋮

$\boxed{8 \times 20 = 160 \text{ bits}}$

size without coding

Message - BCC ABB DDAE CC BBAE DDCC

⇒ Fixed size coding

Character	Count/Frequency		Code
A	8	3/20	000
B	5	5/20	001
C	6	6/20	010
D	4	4/20	011
E	2	2/20	100
	20		

we are using only 5 characters so not 8 bits will be requires, 8 bit is required For 128 symbols

1 have given my own three bit code

1 bit → 0/1

2 bit → 00
01
10
11

- - -
0
to
7

Message - B CC A BB DD AE CC BBAE DDCC
001 010

20×3 = 60 bits

We have to also send table correspond to character for Decoding

5×8 bits — character symbols

5×3 — codes

$40 + 15 = 55$

Msg - 60 bits
Table - 55 bits
115 bits

=> HUFFmann Encoding [Variable size Coding]

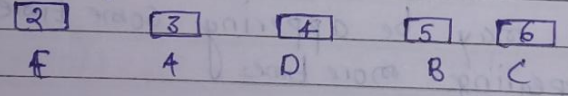
We don't have to take Fixed size Codes. Some character may be appering some time & some may be appearing more time.

If you give small size Code for more appearing character the size of entire message can be reduced

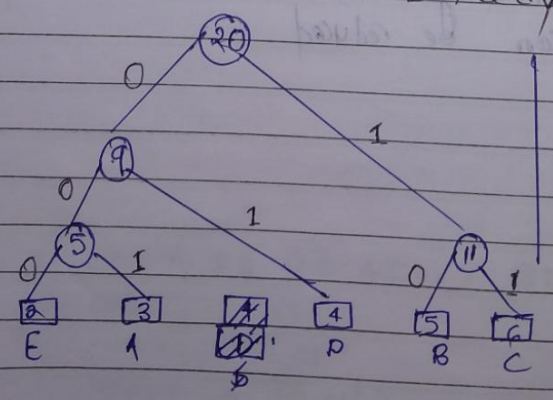
Message - B CC A BB D D A E CC BB A E DD CC

Char	Count	Code	sizes
A	3	001	3×3 = 9 sizes
B	5	10	5×2 = 10
C	6	11	6×2 = 12
D	4	01	4×2 = 8
E	2	000	2×3 = 6
	20		45 bits

step1 → Arrange all alphabets into increasing number of their count



step2 → Optimal Merge Pattern should be Followed on this one [Greedy approach



$\Sigma d_i * f_i$
$= 2\times3 + 3\times3 + 4\times2 + 5\times2 + 6\times2$
$= 45$ bits

step3 → left side edges Mark them as zero & right side edges mark them as 1.

AE DD CC

sizes

3x3 = 9 sizes

5x2 = 10

6x2 = 12

4x2 = 8

2x3 = 6

45 bits

ing number of their count

ld be

dy approach

$= d_i * f_i$

$= 2 \times 8 + 8 \times 3 + 4 \times 2 + 5 \times 2 +$

$= 47$ bits

1

C

then as zero

then as 1.

step 4 → For each alphabets follow a path from root onward

A → 001

B → 10

Message - B CC A BB DD AE CC BB AE DD CC

00 11 11 00 10 01 01

Message size - 45 bits

Along with this message we should also send chart / Tree so that message can be coded.

Alphabes Codes

8 bits x 5 + 12 bits = 52 bits

= 40 bits

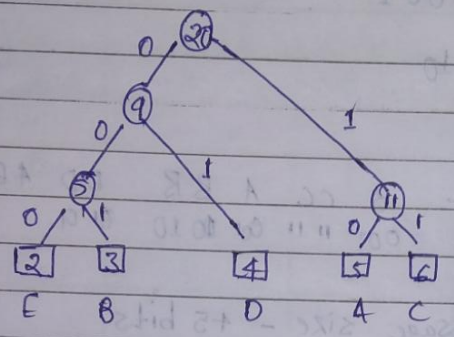
Msg - 45 bits

Tree/ Table - 52 bits

97 bits

Message → B C C D A C C B D A B C C D E A A D A

001 11 11 01 10 11 11 001 01 10 001 11 01



Decode = ? → Do by self

→ Do with help of tree

B C C D - - -

Revise it [As you Face difficulty in datastructure (STL) used in this Q^n]

both are same Problem

Q. N meeting in one room / Activity selection

What is maximum number of meeting can be accomodated in one room.

N = 6

start [] = {1, 3, 0, 5, 8, 5}

end [] = {2, 4, 6, 7, 9, 9}

O/P :- 4

N = 3

start[] = {10, 12, 20}

end [] = {20, 25, 30}

O/P :- 1

(1,2), (3,4), (5,7) and (8,9)

S() =	1	0	3	8	5	8
F() =	2	6	4	9	7	9
	1	2	3	4	5	6

Sort according to Finishing time

(1, 2, 1)
(3, 4, 3)
(0, 6, 2)
(5, 7, 5)
(8, 9, 4)
(8, 9, 6)

*** same Finishing time (तुम चाहो वे कुछ भी करते हो)

if Q^n given that take first one

write First index first

① ③ ⑤ ⑧ end limit = ~~2~~ ~~4~~ ~~7~~ 9

TC → $O(N \log N) + O(N) = O(N \log N)$

sort traverse

SC → $O(N)$

```
struct meeting {
    int start;
    int end;
    int pos;
};
```

*** By Placing static here passed on GFG

```
bool comparator (struct meeting m1, meeting m2)
    IF (m1.end < m2.end) return true;
    else IF (m1.end > m2.end) return False;
    else IF (m1.pos < m2.pos) return true;
    return False;
```

```
void maxMeetings (int s[], int e[], int n) {
    struct meeting meet[n];
    For (int i = 0 ; i < n ; i++) {
        meet[i].start = s[i], meet[i].end = e[i], meet[i].Pos
    }

    Sort (meet, meet+n, comparator);

    vector< int> answer;
    int limit = meet[0].end;
    answer.Push_back (meet[0].pos);

    For (int i=1 ; i<n ; i++) {
        IF ( meet[i].start > limit) {
            limit = meet[i].end;
            answer.Push_back ( meet[i].Pos);
        }
    }

    For (int i=0; i < answer.size() ; i++) {
        cout << answer[i] << " ";
    }
}
```

Q. Minimum Platforms

Given arrival and departure times of all trains that reach a railway station. Find minimum number of platforms required for railway st so that no train is kept waiting

Input : n = 6

arr[] = {0900, 0940, 0950, 1100, 1500, 1800}

dep[] : {0910, 1200, 1120, 1180, 1900, 2000}

O/P :- 3

start = {120, 50, 550, 200, 700, 850}

end = {600, 550, 700, 500, 900, 1000}

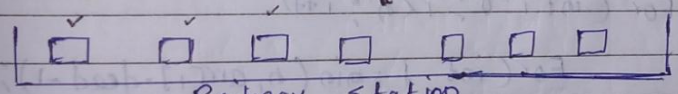
sort both start and end array → Think here well (If train comes then platform is busy if leaves the get empty)

start [] = [50, 120, 200, 550, 700, 850]

end [] = [500, 550, 600, 700, 900, 1000]

see by doing

Plat = 1 2 3
2 3 2
1 2 1 2
max = 1 2 3



Railway station

TC → O(2nlogn) + O(2n)

sorting Two array — Two pointer approach

SC → O(1)

```
int FindplatForm (int arr[], int dept[], int n)
{
    sort (arr, arr+n)
    sort (dep, dep+n);
    int plat_needed = 1, result = 1;
        int i = 1, j = 0;
    while (i < n && j < n) {
        IF (arr[i] <= dept[j]) {
            plat_needed ++;
            i++;
        }
        else IF (arr[i] > dept[j]) {
            plat_needed --;
            j++;
        }
        IF (plat_needed > result)
            result = plat_needed; }
return result;
}
```

Q. Job - Sequencing Problem [each take unit time]

N = 4

Jobs: { (1, 4, 20), (2, 1, 10), (3, 1, 40), (4, 1, 30) }

jobid, max time, Profit

O/P:- 2 60

no of Job Profit

By Putting static

Passed on GFG

```
int mycmp(Job arr1, Job arr
{
    return arr2.Profit < arr1.Pr
}
```

```
Pair <int, int> Jobsheduling (Job arr[], intn)
{
    sort (arr, arr+n, mycmp);
        int Prof = 0;
        int count = 0;
        int A[n] = 0;
    For (int i = 0; i < n; i++)
{
        For (int j = min(n, arr[i].dead - 1); j >= 0
        {
            if (A[j] == 0)
            {
                A[j]++;
                count ++;
                Profit += arr[i].Profit;
                break;
            }
        }
    }
    Pair <int, int> P;
        P = mak_Pair (count, Prof);
    return P;
}
```

Think by this

1	4	50
2	1	10
3	1	40
4	1	30

you always do mistake here

Min^m no. OF Coins needed to make sum

coins[] : [1, 2, 5, 10, 20, 50, 100, 500, 1000] , V=49

V = ~~49~~ ~~29~~ ~~9~~ ~~4~~ ~~20~~

20
20
5
2
2

↓

here greedy Fails

[1, 5, 6, 9] V = 11 → greedy 9, 1, 1] 3

Dynamic

5, 6] 2